

BAŞLIK: KİMLİK DOĞRULAMA, YETKİLENDİRME VE API GÜVENLİĞİ

TÜR: Akademik/Teknik Detaylı Anlatım

BÖLÜM 1: KİMLİK DOĞRULAMA (AUTHENTICATION) VS YETKİLENDİRME (AUTHORIZATION)

Bu iki kavram sürekli birbirine karıştırılır ama tamamen farklı sorulara cevap verirler:

- Kimlik Doğrulama (Authentication - "AuthN"): "Sen kimsin?" sorusuna cevap verir. Bir kullanıcı adı/şifre kombinasyonu, bir API anahtarı veya bir token doğrulanarak kimliğiniz kanıtlanır.
- Yetkilendirme (Authorization - "AuthZ"): "Sen olduğunı kanıtladığın kişi olarak, BU işlemi yapmaya YETKİN var mı?" sorusuna cevap verir. Kimliğiniz doğrulandıktan SONRA devreye girer.

Örnek: Bir e-ticaret API'sinde giriş yaptığınızda kimliğiniz doğrulanır (authentication — "Sen Ayşe'sin"). Ama başka bir kullanıcının siparişini iptal etmeye çalıştığınızda, sistem sizin KİM olduğunuzu biliyor olsa bile bu işlemi REDDEDER (authorization hatası — 403 Forbidden, bkz. 22.2 Bölüm 4) çünkü o siparişin sahibi siz değilsiniz.

BÖLÜM 2: KİMLİK DOĞRULAMA YÖNTEMLERİ

2.1. API Anahtarları (API Keys)

En basit yöntemdir: geliştiriciye sabit, uzun ve rastgele bir metin (örn. sk_live_51H8x...) verilir; her istekte bu anahtar bir başlıkla (genellikle X-API-Key ya da Authorization) gönderilir. Basittir ama zayıf yönleri vardır: anahtar çalınırsa (örneğin yanlışlıkla bir GitHub reposuna commit edilirse) süresiz olarak geçerlidir ve genellikle KİMİN eylemi yaptığını değil sadece HANGİ uygulamanın istek attığını temsil eder (kullanıcı bazlı değil, uygulama bazlıdır).

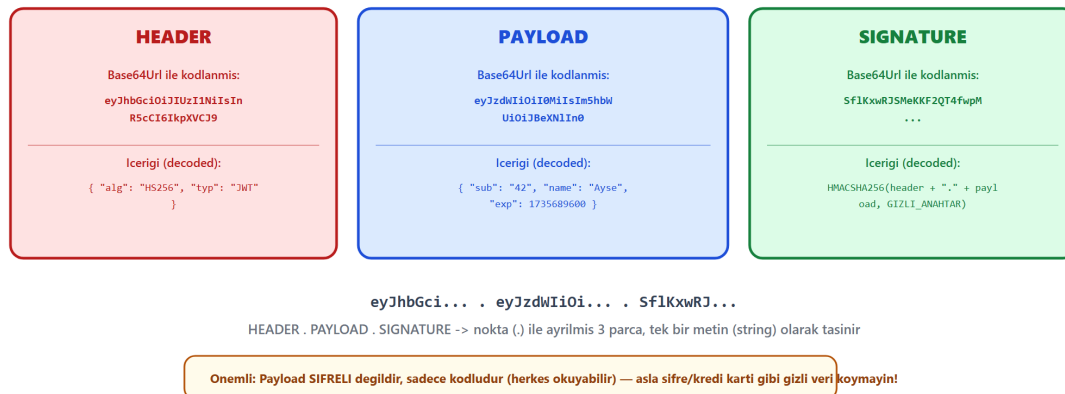
2.2. Temel Kimlik Doğrulama (Basic Authentication)

Kullanıcı adı ve şifre, kullanıcı:sifre formatında birleştirilip Base64 ile kodlanarak Authorization: Basic <kod> başlığında gönderilir. Base64 ŞİFRELEME DEĞİLDİR, sadece kodlamadır — herkes tarafından kolayca çözülebilir! Bu yüzden Basic Auth SADECE HTTPS (TLS ile şifrelenmiş bağlantı) üzerinden kullanılmalıdır; aksi halde şifreniz açık metin gibi ağ üzerinde dinlenebilir.

2.3. Token Tabanlı Kimlik Doğrulama ve JWT

Modern API'lerin çoğu, kullanıcı adı/şifreyi HER istekte tekrar göndermek yerine, bir kere doğrulama yapıp karşılığında geçici bir TOKEN alma yaklaşımını kullanır. En yaygın token formatı JWT'dir (JSON Web Token) — kendi kendini doğrulayan (self-contained), 3 parçadan oluşan bir metindir:

JWT (JSON Web Token) Anatomisi



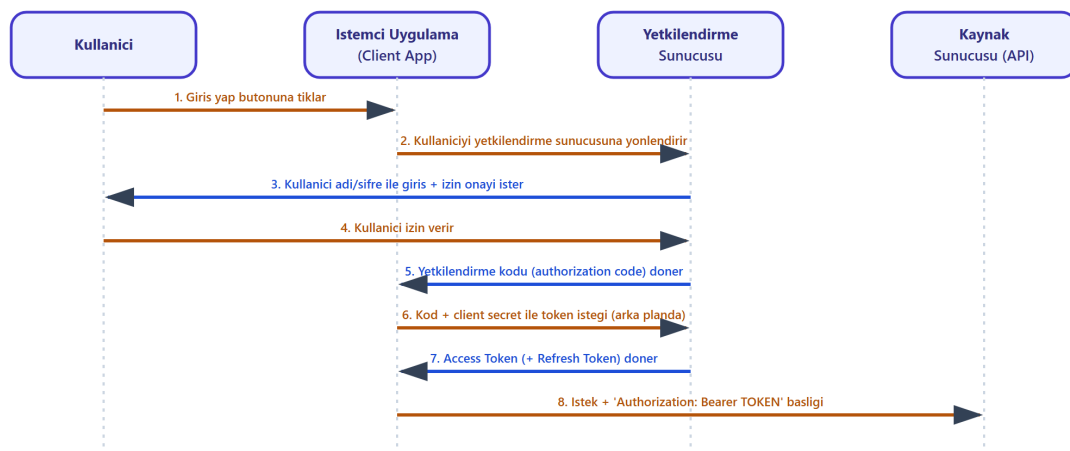
Şekil 1 — Bir JWT, nokta ile ayrılmış Header, Payload ve Signature parçalarından oluşur; Payload şifreli değil sadece kodludur.

JWT'nin en büyük avantajı **SUNUCUNUN DURUM TUTMASINA GEREK KALMAMASIDIR** (stateless): imza (signature) sayesinde sunucu, token'ın kendi ürettiği ve DEĞİŞTİRİLMEDİĞİ garantisini, veritabanına bakmadan, sadece imzayı doğrulayarak alabilir. Bu, HTTP'nin durumsuzluk prensibiyle (bkz. 22.2) birebir uyumludur ve API'nin yatay ölçeklenmesini kolaylaştırır. Dezavantajı: bir token "iptal edilmek" istendiğinde (örn. kullanıcı şifresini değiştirdi), token süresi dolana kadar (exp alanı) hala geçerli kalabilir — bu yüzden JWT'ler genellikle KISA ömürlü (örn. 15 dakika) tutulur ve yanında uzun ömürlü ama iptal edilebilir bir "refresh token" kullanılır.

2.4. OAuth 2.0

OAuth 2.0, "Google ile Giriş Yap" veya "Bu uygulamanın Spotify hesabınıza erişmesine izin ver" gibi senaryolarda kullanılan, YETKİLENDİRME odaklı bir çerçevedir (framework — tek bir protokol değil, birden fazla "akış" - flow - içerir). Temel fikir: kullanıcı ŞİFRESİNİ üçüncü parti uygulamaya PAYLAŞMAZ; bunun yerine güvendiği bir yetkilendirme sunucusu (Google, Facebook, kendi şirketinin kimlik sunucusu) üzerinden, üçüncü parti uygulamaya SINIRLI ve GERİ ALINABİLİR bir erişim izni (token) verir.

OAuth 2.0 Yetkilendirme Kodu Akisi (Authorization Code Flow)



Sekil 2 — Authorization Code akışı: en yaygın ve en güvenli OAuth 2.0 akışı, web uygulamaları için standarttır.

OAuth 2.0'ın birden fazla "akışı" (grant type) vardır: Authorization Code (yukarıdaki şekil — kullanıcı etkileşimi gerektiren web/mobil uygulamalar için), Client Credentials (kullanıcı yok, sadece iki sunucunun

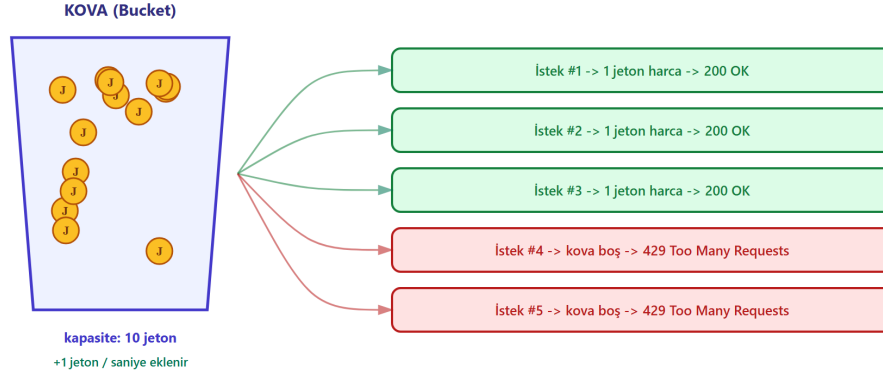
birbirine kimlik doğrulaması yaptığı sunucu-sunucu senaryoları için) ve artık güvensiz kabul edilip terk edilen Implicit Flow. Modülün 22.6 (FastAPI) bölümünde, basitleştirilmiş bir JWT tabanlı kimlik doğrulama akışını uçtan uca kodlayacağız.

BÖLÜM 3: HIZ SINIRLAMA (RATE LIMITING)

Kimlik doğrulanmış (hatta yetkilendirilmiş) bir kullanıcı bile, API'yi kötüye kullanabilir — kasıtlı olarak (saldırı) ya da kasıtsız olarak (hatalı bir döngü, sonsuz retry). Hız sınırlama, bir istemcinin belirli bir zaman diliminde yapabileceği istek sayısını sınırlayarak API'yi ve arkasındaki altyapıyı korur.

Hız Sınırlama (Rate Limiting): Token Bucket Algoritması

Kova, sabit hızda dolar; her istek 1 jeton harcar. Kova boşaldığında istekler reddedilir (429 Too Many Requests).



Şekil 3 — Token Bucket algoritması: en yaygın hız sınırlama yaklaşımlarından biri.

En yaygın algoritmalar:

1. Token Bucket (Jeton Kovası): Sabit kapasiteli bir "kova", sabit bir hızda jetonlarla doldurulur; her istek 1 jeton harcar. Kova boşsa istek reddedilir (429 Too Many Requests). Ani, kısa süreli istek patlamalarına (burst) belirli ölçüde izin verir (kovada biriken jetonlar sayesinde) — bu esnekliği sağladığı için endüstride en yaygın kullanılan algoritmadır.
2. Leaky Bucket (Sızdıran Kova): İstekler bir kuyruğa (kovaya) girer ve SABİT bir hızda işlenir ("sızar"); kova dolarsa yeni istekler reddedilir. Token Bucket'tan farkı, çıkış hızını da sabitleyerek trafiği tamamen düzleştirmesidir (burst'e izin vermez).
3. Sabit Pencere (Fixed Window): "Dakikada en fazla 100 istek" gibi basit bir sayaç tutulur, her dakika sıfırlanır. Basittir ama pencere sınırında (örn. 00:59 ve 01:01'de art arda 100'er istek) istenmeyen patlamalara izin verebilir.
4. Kayan Pencere (Sliding Window): Sabit pencerenin sınır sorununu, son N saniyeyi sürekli kayan bir pencere olarak değerlendirerek çözer; daha adil ama hesaplama açısından biraz daha maliyetlidir.

İyi tasarlanmış bir API, hız sınırı aşıldığında sadece 429 döndürmekle kalmaz, istemciye NE ZAMAN tekrar deneyebileceğini de söyler: Retry-After başlığı ya da X-RateLimit-Remaining / X-RateLimit-Reset başlıkları. 22.5 bölümünde, bu başlıkları okuyup otomatik olarak bekleyen dayanıklı bir istemci yazacağız.

BÖLÜM 4: OWASP API GÜVENLİĞİ TOP 10 (ÖZET)

OWASP (Open Web Application Security Project), API'lerde en sık görülen güvenlik zafiyetlerini düzenli olarak yayınlar. En kritik birkaçı:

1. Kırık Nesne Seviyesi Yetkilendirme (BOLA): En yaygın API zafiyeti. Örnek: GET /siparisler/42 isteğinde, sunucu SADECE token'ın geçerli olduğunu kontrol edip, 42 numaralı siparişin GERÇEKTEN o kullanıcıya ait olup olmadığını KONTROL ETMEZSE, bir kullanıcı sadece ID'yi değiştirerek (43, 44, 45...) başka kullanıcıların siparişlerini görebilir. Çözüm: her kaynağa erişimde "bu kaynak GERÇEKTEN bu

kullanıcıya mı ait?" kontrolü ZORUNLU olmalıdır.

2. Aşırı Veri Açığa Çıkarma (Excessive Data Exposure): API, istemcinin ihtiyacından FAZLA veri döndürüp filtrelemeyi istemciye bırakırsa (örn. şifre hash'i, iç sistem ID'leri gibi hassas alanları da gövdeye koyup "nasılsa frontend göstermez" diye düşünmek), bu veriler tarayıcı geliştirici araçlarıyla kolayca görülebilir.

3. Kaynak Eksikliği ve Hız Sınırlama Yokluğu: Bkz. Bölüm 3.

4. Kırık Kimlik Doğrulama: Zayıf şifreleme, tahmin edilebilir token'lar, süresiz geçerli token'lar.

5. SQL/NoSQL Enjeksiyonu: Kullanıcı girdisinin doğrudan veritabanı sorgusuna eklenmesi (bkz. 15-SQL modülü) — parametrelili sorgular ve ORM kullanımı bu riski büyük ölçüde azaltır.

4.1. HTTPS/TLS Zorunluluğu

Yukarıdaki tüm önlemler, iletişim ŞİFRELENMEMİŞSE (düz HTTP üzerinden) anlamsız hale gelir — aradaki herhangi biri (örn. aynı halka açık Wi-Fi ağındaki bir saldırgan) tüm token'ları, şifreleri ve verileri açık metin olarak okuyabilir ("man-in-the-middle" saldırısı). Bu yüzden production'daki her API HTTPS (TLS ile şifrelenmiş HTTP) kullanılmalı, düz HTTP istekleri otomatik olarak HTTPS'e yönlendirilmeli ya da tamamen reddedilmelidir.

4.2. CORS (Cross-Origin Resource Sharing)

Tarayıcı güvenlik modeli varsayılan olarak, bir web sayfasının BAŞKA bir domain'deki API'ye JavaScript ile istek atmasını engeller ("Same-Origin Policy"). CORS, sunucunun Access-Control-Allow-Origin başlığıyla, HANGİ domain'lerin kendisine istek atabileceğini açıkça beyan etmesini sağlayan bir mekanizmadır. 22.6 bölümünde FastAPI projemizde bu ayarı nasıl yapılandıracağımızı göreceğiz.

SONUÇ

Kimlik doğrulama ile yetkilendirme arasındaki farkı, en yaygın kimlik doğrulama yöntemlerini (API Key, JWT, OAuth 2.0), hız sınırlama algoritmalarını ve en kritik API güvenlik risklerini öğrendik. Artık bir API'yi hem TASARLAMAK (22.3) hem de KORUMAK (22.4) için gereken teoriye sahibiz. Sıradaki iki dosyada, bu teoriyi UYGULAMAYA döneceğiz: önce Python ile var olan bir API'yi dayanıklı şekilde TÜKETMEYİ (22.5), sonra da kendi API'mizi sıfırdan İNŞA ETMEYİ (22.6) öğreneceğiz.